

Cypress

- Up until version 12, Angular used **Protractor** as its default end-to-end testing framework. Protractor was based on WebDriver. Since Angular 12, Protractor is deprecated.
- In new CLI projects, there is no default end-to-end testing solution configured.
- In this section, we will learn to know **Cypress**, a mature end-to-end testing framework.

Installation and configuration:

- Cypress is the product of one company. The test runner we are going to use is open source and free of charge.
- Since Cypress does not use WebDriver, it features a unique architecture. When starting Cypress, a Node.js application launches the browser.
- Cypress is well-maintained and well-documented. With Cypress, you can write valuable end-to-end tests with little effort.

Installation and configuration:

- An easy way to add Cypress to an existing Angular CLI project is the [Cypress Angular Schematic](#).
- In your Angular project directory, run this shell command:

```
ng add @cypress/schematic
```

In the project directory, you will find a sub-directory called cypress. It contains:

- a tsconfig.json file: configuration for all TypeScript files specifically in this directory
- an e2e directory for the end-to-end tests
- A support directory for the end-to-end tests
- A fixtures directory for test data

The test files reside in the e2e directory with extension .cy.ts

The tests itself are structured with the test framework **Mocha**. The assertions (also called expectations) are written using **Chai**.

Jasmine Syntax:

- The test files will use Jasmine's syntax.
- Jasmine has a great syntax for writing tests, with `describe()`, `beforeEach()`, and `it()` as global Jasmine functions. See the following example:

```
describe('... Feature description ...', () => {  
  beforeEach(() => {  
    // Navigate to the page  
  });  
  
  it('... User interaction description ...', () => {  
    // Interact with the page  
    // Assert something about the page content  
  });  
});
```

Cypress Example

As a start, let us write a minimal test that checks the document title. In e2e directory, we create a file: account.cy.ts

```
w5front > cypress > e2e > TS account.cy.ts > ...  
1  describe('Account', () => {  
2    beforeEach(() => {  
3      cy.visit('/account');  
4    });  
5  
6    it('has the correct title', () => {  
7      cy.title().should('equal', 'MyApp');  
8    });  
9  });
```

Cypress Example

Cypress has two shell commands to run the end-to-end tests:

- `npx cypress run`: **Non-interactive test runner**. Runs the tests in a “headless” browser. This means the browser window is not visible.
- `npx cypress open`: **Interactive test runner**. Opens a window where you can select which browser to use and which tests to run. The browser window is visible and it remains visible after completion.
- The second command is typically used in the development environment. You can see the test results the browser window. If you make changes on the test files, Cypress automatically re-runs the tests.
- Both commands require you to start Angular’s development server with `ng serve`

Cypress Example

Cypress provides several ways to find elements. We are going to use the `cy.get` method to find an element by CSS selector.

<https://docs.cypress.io/api/commands/get>

[**https://docs.cypress.io/guides/references/best-practices#Selecting-Elements**](https://docs.cypress.io/guides/references/best-practices#Selecting-Elements)

Cypress Example

In the remainder, let us use take workshop 5 as an example. We will write test file for testing login component:

```
w5front > src > app > login > login.component.html > div.wrapper.fadeInDown
```

Go to component

```
1 <div class="wrapper fadeInDown">
2   <div id="formContent">
3     <!-- Login Form -->
4     <form>
5       <input type="email" name="email" [(ngModel)]="email" class="fadeIn second" placeholder="email">
6       <input type="password" name="password" [(ngModel)]="password" class="fadeIn third" placeholder="password">
7     </form>
8     <button (click)="submit()" class="fadeIn fourth"> submit</button>
9   </div>
10 </div>
11 </div>
```

Cypress Example

The ts file for
the login
component:

```
w5front > src > app > login > TS login.component.ts > [x] BACKEND_URL
1  import { Component, OnInit } from '@angular/core';
2  import { Router } from '@angular/router';
3  import { HttpClient, HttpHeaders } from '@angular/common/http';
4  const httpOptions = {
5    headers: new HttpHeaders({ 'Content-Type': 'application/json'})
6  };
7  const BACKEND_URL = 'http://localhost:8888';
8  @Component({
9    selector: 'app-login',
10   templateUrl: './login.component.html',
11   styleUrls: ['./login.component.css']
12 })
13 export class LoginComponent implements OnInit {
14   email = '';
15   password = '';
16   constructor(private router: Router, private httpClient: HttpClient) { }
17   ngOnInit(): void {}
18   submit(){
19
20     let user = {username: this.email, pwd: this.password};
21     this.httpClient.post(BACKEND_URL + '/login', user, httpOptions)
22       .subscribe((data: any) => {
23         if (data.ok) {
24           alert("correct");
25           sessionStorage.setItem('userid', data.userid.toString());
26           sessionStorage.setItem('userlogin', data.ok.toString());
27           sessionStorage.setItem('username', data.username);
28           sessionStorage.setItem('userbirthdate', data.userbirthdate);
29           sessionStorage.setItem('userage', data.userage.toString());
30           this.router.navigateByUrl("/account");
31         }
32         else { alert("email or password incorrect"); }
33       })
34   }
35 }
36
```

Cypress Example

The
“login.cy
.ts”
in e2e
directory

w5front > cypress > e2e > TS login.cy.ts > ...

```
1 describe('Account', () => {
2   ⚠ beforeEach(() => {cy.visit('/login'); }); // open page http://localhost:4200/login
3   // test invalid username or password
4   it('invalid username or password', () => { // input a username and an invalid password
5     cy.get('input').first().type('k.su@griffith.edu.au');
6     cy.get('input').last().type('123');
7     cy.get('button').click(); // click to submit
8     cy.on('window:alert', (str) => {
9       expect(str).to.equal('email or password incorrect'); // check the alert
10    })
11  });
12  // test invalid username or password
13  it('valid username and password', () => { // input a valid pair of a username and a password
14    cy.get('input').first().type('k.su@griffith.edu.au');
15    cy.get('input').last().type('666666');
16    cy.get('button').click();
17
18    cy.on('window:alert', (str) => {
19      expect(str).to.equal('correct'); // check the alert
20    })
21
22    cy.url().should('include', '/account') // check the current page url
23    // check the sessional storage
24    cy.getAllSessionStorage().then((result) => {
25      expect(result[http://localhost:4200/]['username']).to.deep.equal('k.su@griffith.edu.au')
26    })
27  });
28 });
```

Reading materials

1. Jasmine's [GitHub page](#)
2. <https://testing-angular.com/end-to-end-testing/#end-to-end-testing>
3. <https://www.cypress.io/>